GREEN BOLSAS

Guia de defesa e referência técnica

Portal de oportunidades académicas, candidaturas, subscrições Premium e mentoria

Documento para estudantes, programadores e apresentação perante banca.

Baseado nos ficheiros do projeto e na expansão implementada. Distingue funcionalidades no código de passos de instalação e validação ainda pendentes.

Versão de referência: 05 de novembro de 2026

1. O projeto em poucas palavras

A narrativa recomendada para começar a defesa.

Green Bolsas é um portal web para divulgar bolsas de estudo comparticipadas e apoiar o percurso do estudante desde a descoberta da oportunidade até à candidatura e, para assinantes Premium, à orientação por um mentor.

A plataforma tem três perfis: administrador, estudante e mentor. O administrador gere a operação; o estudante consulta oportunidades, candidata-se, acompanha a conta e pode escolher mentores com Premium activo; o mentor recebe pedidos, aceita ou recusa e conversa com estudantes ligados.

O sistema mantém a arquitectura existente - HTML, CSS, JavaScript, PHP e MySQL/MariaDB - e foi ampliado sem substituir a aplicação por outro framework.

Proposta de valor: reunir oportunidades, candidaturas e acompanhamento numa plataforma com permissões e histórico, em vez de deixar esses passos dispersos.

Problema que o portal procura resolver

- O estudante precisa de encontrar oportunidades actualizadas e perceber os prazos e requisitos.
- A candidatura precisa de ficar associada à conta e ter um estado consultável.
- A orientação individual precisa de um fluxo controlado: Premium activo, pedido enviado, aprovação do mentor e só então chat.
- A administração precisa de dados reais para gerir bolsas, pessoas, mentores, candidaturas e subscrições.

O que deve ser dito com precisão

Os pagamentos são demonstrativos e não cobram dinheiro. A aplicação e a migração foram verificadas sintacticamente, mas a migração e os fluxos com a base de dados ainda precisam de ser executados no ambiente local antes de afirmar que a instalação foi validada de ponta a ponta.

2. Perfis e permissões

O servidor confirma as permissões em cada operação protegida.

Perfil	Pode fazer	Limites principais
Visitante	Consultar, pesquisar e filtrar bolsas publicadas e dentro do prazo.	Não se candidata sem iniciar sessão.
Estudante básico	Gerir dados pessoais, guardar bolsas, candidatar-se, consultar estado/histórico e receber notificações.	Não consulta nem escolhe mentores sem Premium activo.
Estudante Premium	Tudo do básico; consultar mentores, ver compatibilidade, pedir ligação e conversar após aceitação.	Premium tem prazo; expirado bloqueia novas mensagens, preservando o histórico em leitura.
Mentor	Consultar perfil, solicitações e estudantes ligados; aceitar/recusar e conversar nas ligações autorizadas.	Não gere bolsas, pagamentos, subscrições ou utilizadores; não vê estudantes alheios.
Administrador	Gerir bolsas, utilizadores, mentores, candidaturas, subscrições e notificações; consultar indicadores.	Acesso verificado pela API com is_admin. Não há registo público de administradores.

Como o papel é determinado

A tabela users mantém o indicador is_admin do sistema original. Se is_admin=1, a API apresenta o papel admin. Caso contrário, usa o campo role, com student como padrão e mentor atribuído apenas durante o registo administrativo do mentor. O formulário público de registo cria sempre estudantes; o navegador não escolhe o próprio papel.

Regra para a defesa: ocultar um botão melhora a interface, mas não é segurança. O servidor também verifica identidade e papel antes de executar a acção.

Criar a primeira conta de administrador

Crie primeiro uma conta normal com o email do administrador. Depois, numa sessão autorizada do phpMyAdmin, na base bolsas, execute:

```
UPDATE users SET is_admin=1 WHERE email='admin@exemplo.ao';
```

Não existe uma conta administrativa pré-criada. Confirme que a consulta alterou uma linha; termine e volte a iniciar sessão para carregar o papel actualizado.

3. Fluxo geral do programa

Do pedido no browser até à persistência no MySQL.

1. Browser index.html + assets/app.js	2. API api/index.php	3. Base de dados MySQL/MariaDB
Formulário, filtros, painéis e chat.	Valida sessão, CSRF, dados e permissões; devolve JSON.	Guarda contas, bolsas, candidaturas e eventos.

Exemplo: candidatura a uma bolsa

- O JavaScript envia JSON para api/index.php?action=apply.
- A API exige sessão e valida a carta de motivação, a bolsa e o prazo.
- Uma instrução preparada grava a candidatura; uma chave única impede candidaturas repetidas à mesma bolsa.
- A API regista o estado inicial no histórico e gera uma notificação para o estudante.
- O painel do estudante apresenta bolsa, instituição, país, prazo, estado e histórico.

Fluxo de sessão

A sessão PHP guarda o identificador do utilizador. O endpoint me devolve o utilizador autenticado, preço Premium e token CSRF. O frontend usa o token em pedidos POST. A API volta a consultar a conta na base, em vez de confiar no papel enviado pelo browser.

Tecnologias e responsabilidade de cada camada

Tecnologia/ficheiro	Responsabilidade
HTML - index.html	Estrutura da página, cabeçalho, áreas dos painéis e diálogo modal.
CSS - assets/style.css	Identidade visual responsiva, cartões, formulários, tabelas e estados.
JavaScript - assets/app.js	Eventos da interface, pedidos fetch, renderização e mensagens.
PHP - api/index.php	Autenticação, validação, controlo de acesso, regras de negócio e JSON.
MySQL/MariaDB	Persistência relacional, chaves estrangeiras, índices e unicidade.

4. Percurso do estudante

Consulta, candidatura, perfil e acompanhamento.

1. Explorar oportunidades

- A página lista apenas bolsas activas, publicadas e com prazo não vencido.
- Filtros por país, âmbito, nível e pesquisa textual reduzem a lista.
- O cartão mostra instituição, destino, nível, comparticipação e data limite.
- Uma bolsa pode ser guardada e removida dos favoritos; esta lista é pessoal.

2. Candidatar-se

- O visitante é convidado a criar conta ou iniciar sessão.
- A carta de motivação tem no mínimo 30 caracteres.
- A API confirma que a bolsa existe, está publicada/activa e dentro do prazo.
- A restrição única (user_id, scholarship_id) impede registo duplicado.
- O estudante pode retirar candidaturas em estado submetida ou em_analise; a retirada também fica no histórico.

Estados de candidatura

Estado interno	Significado para apresentar
submetida	Recebida pela plataforma.
em_analise	Em avaliação.
aceite	Aceite.
recusada	Não aceite.
retirada	Retirada pelo estudante.

3. Perfil pessoal

O perfil permite actualizar nome, telefone, país, cidade, data de nascimento, instituição, formação e nível académico. O email é apresentado como não editável no formulário actual. Não há fotografia de perfil nem secção de definições.

Candidaturas antigas continuam nas tabelas existentes. A migração amplia o histórico e os campos sem apagar os registos anteriores.

5. Premium, mentorias e chat

O fluxo com mais regras de negócio.

Compra demonstrativa

- O estudante escolhe um método demonstrativo: Multicaixa Express simulado, transferência simulada ou referência fictícia.
- O servidor cria uma referência DEMO, regista o método e activa Premium por 30 dias.
- Não existe ligação a banco, operadora, gateway ou transferência real.
- O servidor compara premium_until com a data actual e altera o plano para básico quando expira.

Pedido e aprovação de mentoria

- Só um estudante com Premium activo pode ver a lista e pedir uma ligação.
- O mentor tem de existir, estar registado pela administração e estar activo.
- O estudante pode ter apenas um pedido pendente ou uma ligação aceite de cada vez. A API serializa o pedido com bloqueio da linha do estudante.
- O mentor recebe notificação e aceita ou recusa.
- Aceite desbloqueia o chat; rejeitado permite ao estudante escolher outro mentor.
- O estudante pode terminar a ligação. A relação e as mensagens ficam preservadas para histórico.

Estados da ligação

Estado	Transição / efeito
pending	Pedido enviado; sem chat.
accepted	Aprovado; chat permitido enquanto Premium e mentor activos.
rejected	Recusado pelo mentor; novo pedido pode ser feito.
cancelled	Pedido pendente cancelado pelo estudante.
ended	Mentoria terminada pelo estudante; histórico preservado.

Chat e expiração

A API só devolve mensagens à pessoa estudante ou mentora que participa naquela ligação. Mensagens são carregadas por consulta periódica enquanto o diálogo está aberto. O envio exige ligação aceite, Premium ainda válido e mentor activo. Quando um destes critérios falha, a leitura do histórico fica disponível em modo de leitura.

6. Como a recomendação de mentores funciona

Pontuação determinística e explicável.

O estudante pode seleccionar uma bolsa real para comparar com os dados de cada mentor. O servidor compara quatro atributos: país, nível académico, área de formação e idioma. Cada coincidência encontrada soma 25 pontos percentuais. O sistema apresenta os critérios coincidentes e ordena a lista pela pontuação.

Critério	Dados comparados
País	País da bolsa e países de experiência do mentor.
Nível	Nível da bolsa e níveis/experiência do mentor.
Área	Área da bolsa e áreas, especialidades ou experiência do mentor.
Idioma	Idioma da bolsa e idiomas indicados pelo mentor.

Limites de interpretação

- O resultado é um auxílio à decisão, não uma previsão de sucesso da candidatura.
- A comparação actual é textual: procura correspondência do valor da bolsa nos campos registados pelo administrador.
- Se não houver bolsa seleccionada, não aparece percentagem; a lista mostra mentores activos sem atribuir compatibilidade fictícia.
- A pontuação actual usa quatro critérios fixos de 25%. Valores ausentes não são renormalizados. Explique esse detalhe se a banca perguntar como foi calculada a percentagem.

Exemplo para a banca: se país, nível e idioma coincidem, a compatibilidade apresentada é 75%, e o sistema identifica esses três critérios. Não existe aleatoriedade.

7. Notificações e eventos

Eventos ligados a operações reais.

A tabela notifications guarda destinatário, tipo, chave do evento, conteúdo, link, estado de leitura e data. A combinação utilizador + event_key é única; a inserção com INSERT IGNORE evita duplicados para o mesmo evento.

Evento	Destinatário / resultado
Nova bolsa publicada	Todos os utilizadores activos.
Prazo a 7, 2 ou 0 dias	Todos os utilizadores activos; bolsa identificada.
Candidatura submetida	Estudante que se candidatou.
Estado de candidatura alterado	Estudante dono da candidatura.
Pedido de mentoria	Mentor escolhido.
Pedido aceite ou recusado	Estudante que pediu a ligação.
Nova mensagem	Outro participante da ligação.
Premium activado / perto do fim / expirado	Estudante titular.

Verificação diária de prazos

api/cron_notifications.php corre em PHP CLI. O LEIAME.md descreve como agendá-lo diariamente no Agendador de Tarefas do Windows. Além do agendamento, o site verifica avisos ao abrir a aplicação. O agendamento diário ainda precisa de ser configurado na máquina.

Acções na central

Consultar o histórico, ver o contador de não lidas, marcar uma ou todas como lidas e apagar uma notificação própria. A área administrativa consegue consultar os avisos globais e pessoais.

8. Painel administrativo

Indicadores extraídos de dados persistidos.

O painel obtém os totais directamente das tabelas: utilizadores, bolsas, bolsas activas, bolsas encerradas, candidaturas, contas Premium, mentores e ligações aceites. A actividade recente agrega criação de contas, candidaturas, pagamentos simulados e pedidos de mentoria.

Área	Operações disponíveis
Resumo	Indicadores e actividade recente.
Bolsas	Criar, editar, publicar, guardar rascunho e suspender. A suspensão preserva referências históricas.
Utilizadores	Pesquisar por nome/email, consultar perfil e associação a mentor, activar ou suspender contas.
Mentores	Criar conta e perfil, editar dados, activar/desactivar, consultar ligações e histórico.
Candidaturas	Consultar candidaturas e alterar estado; cada alteração é registada.
Subscrições	Consultar plano, estado, datas e pagamentos/referências demonstrativos.
Notificações	Consultar eventos e destinatários.

Gestão de bolsas

O endpoint antigo admin_publish continua protegido. O endpoint ampliado admin_scholarship_save permite estado published, draft ou suspended. A acção apresentada como eliminar suspende a bolsa e desactiva-a, preservando candidaturas e chaves estrangeiras.

Conta de mentor

Só a API administrativa cria utilizadores com role mentor. A criação de utilizador e perfil é feita numa transacção, para evitar uma conta sem perfil se uma inserção falhar.

9. Modelo de dados

Tabelas centrais e relações principais.

Tabela	Papel no modelo
users	Conta, credenciais hash, is_admin, role, estado, dados pessoais e validade/estado Premium.
scholarships	Oportunidade, entidade, país, âmbito, nível, percentagem, prazo, descrição, estado, área e idioma.
applications	Relação estudante-bolsa, motivação, estado e data; uq_app impede duplicação.
application_history	Mudanças de estado, autor e data.
payments	Histórico demonstrativo, montante, referência, método e estado.
mentor_profiles	Biografia, formação, experiência, países, áreas, níveis, idiomas, especialidades e disponibilidade.
mentor_connections	Pedido/ligação entre estudante e mentor, estado, bolsa associada e datas.
mentor_messages	Mensagens associadas a uma ligação, remetente, data e leitura.
notifications	Inbox por utilizador e deduplicação por chave do evento.
saved_scholarships	Favoritos do estudante; chave primária composta utilizador + bolsa.
mentorship_requests	Tabela legada do fluxo inicial. Preservada para dados existentes; novos pedidos usam mentor_connections.

Relações essenciais

- Um utilizador pode possuir várias candidaturas; cada candidatura aponta para uma bolsa.
- Uma ligação referencia um estudante e um mentor, ambos em users, e pode apontar para uma bolsa.
- Mensagens dependem da ligação; notificações pertencem ao destinatário.
- Chaves estrangeiras e políticas RESTRICT/SET NULL/CASCADE protegem histórico ou removem dependentes quando isso é apropriado.
- A migração é incremental: adiciona colunas e tabelas; não recria a base nem elimina os dados anteriores.

As tabelas users, scholarships, applications, mentorship_requests e payments existiam antes da expansão. As novas tabelas dão suporte à separação de papéis, perfil, subscrições, chat, notificações e histórico.

10. Segurança e lógica no servidor

O que dizer quando perguntarem sobre acessos indevidos.

Controlo	Aplicação no projeto
Sessão	Sessão PHP; cookie HttpOnly e SameSite=Lax; ID regenerado ao autenticar.
CSRF	Token aleatório por sessão devolvido em me e exigido no cabeçalho de todos os pedidos POST.
Autorização	Funções need(), admin(), mentor() e premium() verificam sessão, papel e validade.
SQL	Consultas parametrizadas para dados variáveis; exceções PDO não expõem detalhes do banco ao utilizador.
Chat privado	A consulta exige que o utilizador seja um dos dois participantes da ligação.
Conteúdo HTML	Frontend escapa os valores antes de os inserir em templates HTML.
Integridade	Chaves estrangeiras, índices e chaves únicas reduzem duplicação e registos órfãos.

Exemplos de defesa no servidor

- Um estudante não consegue publicar bolsa ao enviar directamente o pedido admin_publish: a API responde 403 se is_admin não estiver activo.
- Um estudante básico não consegue escolher mentor: a rota exige Premium vigente no banco.
- Um terceiro não consegue abrir uma conversa alterando connection_id: a query também exige ser participante.
- Um utilizador não consegue declarar is_admin ou role no cadastro: o cadastro define role=student no servidor.
- Uma sessão com conta suspensa deixa de ser aceite.

Limites de segurança antes de produção

A instalação local ainda precisa de HTTPS, configuração segura do cookie Secure em produção, política de cópias de segurança, limitação de tentativas de autenticação, monitorização e revisão completa de segurança. A simulação de pagamentos não deve ser tratada como pagamento real.

11. Instalação e operação local

Passos para preparar a aplicação para demonstração.

Pré-requisitos

- Apache e MySQL/MariaDB iniciados no XAMPP.
- PHP 8 ou superior com pdo_mysql activo. O PHP incluído no XAMPP foi verificado com esse driver.
- Base de dados bolsas importada e credenciais locais correctas em api/config.php.
- Projecto servido por Apache em http://localhost/Green-bolsas/; não abrir index.html como file://.

Ordem de preparação

Passo	Acção
1	Iniciar Apache e MySQL no XAMPP.
2	Importar database/schema.sql apenas numa instalação nova. Se a base já existe, não reimportar às cegas para não duplicar dados de exemplo.
3	Executar uma vez database/migration_expansion.sql na base bolsas.
4	Confirmar host, nome da base, utilizador e palavra-passe em api/config.php.
5	Criar uma conta estudante e promover uma conta administrativa através de is_admin, se ainda não existir.
6	Entrar como administrador e criar uma conta mentor de teste.
7	Configurar tarefa diária para api/cron_notifications.php, usando o PHP CLI do XAMPP.
8	Abrir http://localhost/Green-bolsas/ e executar os cenários de demonstração.

Consulta de contagem de utilizadores

```
SELECT COUNT(*) AS total_usuarios FROM users;
```

Não há um número fixo de utilizadores no código. O total depende dos registos presentes na base local.

12. Mapa dos ficheiros

Onde procurar cada comportamento.

Ficheiro	Conteúdo / responsabilidade
index.html	Markup da página principal e regiões dos painéis.
assets/app.js	Estado do frontend, navegação, formulários, chamadas fetch e renderização de cada perfil.
assets/style.css	Estilos responsivos, paleta, cartões, grelhas, formulários e tabelas.
assets/favicon.svg	Ícone da aplicação.
api/index.php	API JSON, sessão, autenticação, autorização, candidaturas, bolsa, Premium, mentoria, chat, notificações e gestão administrativa.
api/config.php	Configuração de acesso ao MySQL e preço mensal demonstrativo. Não partilhar credenciais.
api/cron_notifications.php	Verificação diária via PHP CLI de prazos e expiração de subscrição.
database/schema.sql	Esquema original e bolsas de demonstração da instalação inicial.
database/migration_expansion.sql	Migração incremental para funções acrescentadas.
LEIAME.md	Arranque local, migração, tarefa diária e lembretes de operação.

Como depurar um pedido

- Primeiro inspeccione o evento no assets/app.js e o nome action enviado.
- Localize o case correspondente em api/index.php.
- Confirme a sessão, a regra de autorização e a validação antes da consulta SQL.
- Confira a estrutura da tabela na migração e a configuração do banco.
- A API devolve JSON; o frontend apresenta error.message e o estado HTTP.

Convenção de API

As leituras são pedidos GET e as alterações são POST com JSON e cabeçalho X-CSRF-Token. A maioria das rotas protegidas responde com 401 sem sessão, 403 sem permissão, 404 para registo inexistente e 409 quando a operação conflita com o estado actual.

13. Roteiro de demonstração

Uma sequência curta que demonstra os principais requisitos.

Antes de começar

- Inicie Apache e MySQL; confirme a migração aplicada.
- Prepare três contas: administrador, estudante e mentor. O Premium do estudante pode ser activado pelo fluxo demonstrativo.
- Prepare uma bolsa futura, publicada, com país, nível, área e idioma compatíveis com os dados do mentor.
- Abra uma janela privada para trocar de perfil sem confundir cookies/sessões.

Demonstração em 6 minutos

Tempo	Acção e frase-guia
0:00-0:40	Explorar a página pública. “O catálogo é filtrável e mostra prazos e comparticipação.”
0:40-1:30	Entrar como estudante, guardar uma bolsa e candidatar-se. “A candidatura é persistida e recebe estado/histórico.”
1:30-2:10	Abrir perfil e candidaturas. “Os dados apresentados pertencem à sessão autenticada.”
2:10-2:50	Activar Premium demonstrativo. “A referência é fictícia e o servidor activa validade de 30 dias.”
2:50-3:40	Escolher uma bolsa na recomendação e consultar mentores. “A percentagem vem de quatro comparações reais, não de aleatoriedade.”
3:40-4:30	Enviar pedido; mudar para mentor; aceitar. “O pedido fica pendente e o chat só abre após a decisão.”
4:30-5:10	Enviar mensagens e terminar ligação. “Só os participantes consultam o histórico.”
5:10-6:00	Entrar como administrador, mostrar estatísticas, utilizadores, mentores e candidaturas. “O painel agrega dados da base.”

Plano alternativo

Se a banca não permitir acesso à base, demonstre a instalação e use capturas de ecrã previamente produzidas. Não invente contagens: mostre consultas SQL reais ou declare que o banco não está disponível.

14. Texto de apresentação sugerido

Pode adaptar ao tempo e à linguagem da banca.

“O Green Bolsas é uma plataforma para organizar oportunidades de estudo, candidaturas e acompanhamento por mentores. Mantivemos a aplicação existente em PHP, JavaScript e MySQL e ampliámos a solução com papéis, notificações, favoritos, subscrições demonstrativas e chat ligado a mentorias aprovadas.”

“O visitante pode explorar as bolsas publicadas. Para se candidatar, cria uma conta de estudante; a aplicação valida a carta, o prazo e impede candidaturas duplicadas. O estudante acompanha o estado e o histórico na sua área pessoal.”

“A mentoria exige uma subscrição Premium activa. O pagamento é simulado e identificado como demonstração. O estudante escolhe um mentor através de critérios transparentes, envia um pedido e aguarda aceitação. Só depois da aprovação é que o chat fica disponível. As conversas são privadas aos dois participantes e permanecem consultáveis quando a subscrição expira ou a ligação termina.”

“O administrador tem acesso exclusivo à gestão de bolsas, utilizadores, mentores, candidaturas, subscrições e notificações. O controlo é feito na API, não apenas escondendo botões no ecrã. A base é relacional e usa chaves estrangeiras, índices e chaves únicas para manter integridade.”

“Como evolução, a instalação precisa de validação de ponta a ponta depois da migração, configuração do agendamento diário e, antes de produção, HTTPS, gateway de pagamento real e reforço operacional. O objectivo desta fase foi consolidar os fluxos demonstrativos sem apagar dados existentes.”

Ajuste importante

Use “implementado no código” para funcionalidades criadas mas ainda não executadas com o banco. Só use “testado” depois de concluir um fluxo real com a base migrada.

15. Perguntas prováveis da banca

Respostas directas, sem prometer além do implementado.

Como impedem que um estudante publique uma bolsa?

A API chama a verificação admin() e exige is_admin. A interface também não apresenta a gestão ao estudante, mas a autorização decisiva é no servidor.

O pagamento Premium é real?

Não. É um modo demonstrativo: cria referência DEMO e activa o plano localmente; não contacta instituição financeira.

Como sabem que só o estudante e mentor acedem ao chat?

A consulta da conversa verifica a ligação e exige que o utilizador actual seja student_id ou mentor_id. O envio exige ainda ligação aceite, Premium vigente e mentor activo.

Como é calculada a compatibilidade?

Quatro critérios textuais, cada um vale 25 pontos: país, nível, área e idioma. A resposta também expõe os critérios coincidentes.

O que acontece quando o Premium expira?

A API actualiza o plano para básico, bloqueia novas mensagens e pedidos, mas preserva o histórico em modo de leitura.

Como evitam candidatos duplicados?

Índice único na combinação utilizador-bolsa, além da verificação de disponibilidade e sessão.

Onde ficam as notificações?

Na tabela notifications, por destinatário e event_key única para evitar duplicação. O contador soma as não lidas.

Quantos utilizadores existem?

Não é um valor fixo no código. Consultamos SELECT COUNT(*) FROM users na base instalada.

O que falta validar?

Aplicar a migração no MySQL local, executar testes de ponta a ponta com os três papéis e configurar o agendamento diário. A revisão sintáctica não substitui esses testes.

16. Estado da implementação e checklist final

Transparência para uma defesa credível.

Implementado no código

- Painéis e rotas para administrador, estudante e mentor.
- Regras de cadastro, publicação, candidatura, suspensão, Premium, aprovação de mentoria e chat privado.
- Perfil sem fotografias, favoritos, histórico de candidaturas e notificações por evento.
- Migração incremental, script de avisos diários e instruções no LEIAME.md.
- Validação sintáctica do PHP e JavaScript e verificação de espaços no diff.

Ainda depende da instalação e teste no MySQL

- Executar database/migration_expansion.sql na base bolsas.
- Iniciar o MySQL e confirmar as credenciais em api/config.php.
- Testar cadastro/login, estudante básico, Premium, mentor, administrador, acessos negados e preservação de dados.
- Agendar api/cron_notifications.php uma vez por dia no Windows.
- Confirmar que o histórico da instalação anterior permanece visível depois da migração.

Ainda não faz parte do sistema

- Pagamento financeiro real ou confirmação por webhook.
- Mensagens em tempo real por WebSocket; actualmente o chat actualiza por consulta periódica enquanto está aberto.
- Envio de email ou SMS; notificações são internas ao portal.
- Apagamento físico de bolsas; a gestão usa suspensão para preservar histórico.
- Revisão formal de segurança e testes de integração após a migração.

Critério de conclusão para a apresentação: a migração executada, o banco acessível e pelo menos um cenário completo demonstrado para administrador, estudante e mentor.

Referências no repositório: index.html, assets/app.js, assets/style.css, api/index.php, api/config.php, api/cron_notifications.php, database/schema.sql, database/migration_expansion.sql e LEIAME.md.